AI Native Engineering Practices

林肖维 | AI 驱动的软件开发、Agent 协作与工程实践

AI 不只是代码补全工具，而是参与需求、设计、实现、验证与迭代的工程协作者。

01 模型、Prompt 与 Context 管理

- **模型选择：**根据需求、质量、速度、Token 效率、上下文与工具能力选择合适模型。
- **Model-specific Prompting：**针对不同模型调整 Prompt、约束与反馈方式；例如使用 GPT-5.6 Sol 时明确限制不必要的过度防御性代码、fallback、重复校验与过度抽象。
- **模型切换：**Claude / Codex / ChatGPT 组合使用；模型陷入局部思路时切换模型、角色或重置上下文。
- **Context Engineering：**只提供任务真正相关的信息，减少无关历史、Token 浪费与上下文污染。
- **上下文压缩：**通过 Summary、Task Package、Knowledge 保留关键状态，支撑长任务持续执行。

02 AI 全流程开发

- **需求与设计：**AI 参与需求澄清、Brainstorming、边界分析、方案设计与技术选型。
- **Human-led Architecture：**AI 辅助方案探索；CUDA/GPU、高并发、异步架构等关键技术路线仍由工程师结合性能、成本与稳定性判断并引导。
- **开发与验证：**覆盖 Coding、Debug、Test、Review、Refactor 与 Documentation。
- **Plan Before Code：**复杂任务先理解代码库、明确影响范围与实现计划，再进入编码。
- **Quality Gate：**结合测试、Diff Review、静态检查与人工判断验证 AI 输出。

03 Multi-Agent 协作与编排

- **Agent 分工：**Research / Developer / Reviewer / Tester 等不同角色协作。
- **并发执行：**独立研究、开发与 Review 任务并行处理，提高整体吞吐。
- **任务接力：**通过 Goal、Progress、Decisions、Artifacts、Next Actions、Handoff Skill 跨 Agent / Session 接力。
- **统一编排：**管理模型选择、任务拆分、依赖关系、并发与最终结果合并。

04 多模型验证与 Skills

- **Cross-Model Review：**关键架构、核心代码和研究结论使用不同模型与针对性 Prompt，进行多模型、多轮次交叉验证。
- **Independent Reviewer：**实现者与 Reviewer 分离，从独立视角发现错误假设与遗漏。
- **Reusable Skills：**沉淀 Brainstorming、Grill、Review、Handoff 等高频工作流。
- **Coding Harness：**合理使用 Goal / Advisor 等 Harness 能力，提高规划、执行与结果质量。

05 AI 工程化与长期维护

- **MCP / Tool Calling：**让 Agent 操作代码、文件、终端、API 等真实开发环境。
- **Knowledge Management：**沉淀架构决策、项目经验和已验证知识，而非依赖完整聊天历史。
- **Delete / Simplify / Refactor：**主动削减重复代码、过度抽象与功能膨胀，保持系统长期可维护。
- **Risk Control：**低风险任务提高自动化程度；核心代码、权限、数据库操作保留人工确认。
- **Continuous Improvement：**通过 Improvement Proposal 持续优化 Agent、Skills、工具与协作流程。

06 AI × 量化研究与交易实践

- **Fundamental & Equity Research：**AI 辅助企业经营/财务数据、公告、财报、研报与个股分析，提炼核心假设、催化剂与风险。
- **Quant Research Copilot：**结合宏观、因子、期权与市场微观结构开展特征构建、信号挖掘、策略研究与实验分析。
- **Research Integrity & Logic：**辅助识别未来函数 / Look-ahead Bias、数据泄漏与过拟合；挑战论证链、搜索反例，并以多模型复核研究假设。
- **Strategy, Capital & Risk：**AI 参与策略逻辑优化、策略设计、回测与实盘迭代，并辅助仓位、资金分配、风险预算与组合约束设计。
- **Performance Engineering：**辅助算法/计算复杂度分析与优化，并结合多核 CPU / CUDA 并行加速计算密集型任务。
- **Agent Research Workflow：**通过 Task Package、Knowledge、Handoff、Improvement Proposal 组织长期研究、任务接力与平台迭代。

核心理念 AI 是参与需求、研究、设计、实现、验证与迭代的工程协作者；关键架构、技术取舍与最终责任仍由工程师负责。

More context ≠ Better context • **More code ≠ Better code** • **More agents ≠ Better system**

Representative practice: Claude · Codex · ChatGPT · MCP · Skills · Multi-Agent · Full-stack · AI x Quant

AI Native Engineering = Model Selection + Prompt Adaptation + Context Engineering + Agent Orchestration + Human-led Architecture + Quality Control